SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO1 - ASSESSMENT TOOL - 1

DESCRIPTIVE WRITTEN TEST

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 1: Descriptive Written Test**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO1 Statement	*Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.* **(BL3)**		
Student Name	**M.DATHU**	**Reg. No.**	**192311158**
Section/Batch	**Slot A /2023**	**Date**	**21-10-2026**

Code of Conduct:

I **M.DATHU 192311158** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.DATHU

10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Applying Software Testing as an Engineering Activity

Introduction:

Software testing is a systematic process used to identify defects and verify wheter software meets user requirements. testing should be planned and organized rather than performed randomly. A structured testing process improve software quality.

Problem with Random Testing:

1. Incomplete testing: Important features may remain untested
2. Missed defects: Invalid inputs and boundary conditions may be overlooked.
3. Time wastage: Similar tests may be repeated unnecessarily.
4. Unclear result: Expected output may not be defined.
5. Difficult debugging: Defects may be difficult to reproduce.
6. Increased cost: late defect detection requires more effort to fix

Redesigning the testing process

Step: 1 Requirement Analysis
Identify functional and non-functional requirements to determine what needs to be tested.
Step: 2 Test Planning
Define testing objectives, scope, resources, schedule, and responsibilities.
Step: 3 Test case design
Prepare test cases containing inputs, execution steps, preconditions, and expected result.
Step: 4 Test environment setup
configure the required hardware, software, database and test data.
Step: 5 Test execution
Execute the cases and compare actual result with expected result.
Step: 6 Defect Reporting
Record defect, correct them and repeat failed tests to verify the fixes
Perform regression testing to ensure existing features still work.
Step: 7 Test closure
Evaluate test coverage unresolved defects, and overall software quality before release.

4. Importance of structured test case design

structured test case design is the process of preparing test cases sistematically based on requirements inputs and expected output.

1. Improves coverage: Ensure that important requirements are tested
2. Early defect detection: Identifies errors before software releases.

3. Avoids duplications: Reduce unnecessary repeated testing.
4. Ensures repeatability: Allow tests to be executed consistently.
5. Improves traceability: links test cases to specific requirements.
6. Reduce costs: minimize rework and maintenance expenses.
7. Support automation: Enables repetitive tests to be automated.
8. Improve reliability: Reduces software failures and increases customer satisfaction.

Example of structured Test case design

Test case 1: Enter a valid username and password. Expected result: login successful.

Test case 2: Enter an incorrect password. Expected result: An error message is displayed

Test case 3: enter an invalid username. Expected result: login is rejected

Test case 4: leave both field empty: expected result: validation messages are displayed

Test case 5: Enter SQL injection-like input. Expected result authentication bypass is prevented.

Techniques to Enhance Test effectiveness

1- Equivalence Partitioning

This technique divides input data into valid and invalid groups and selects representative values from each group.

Example: For a valid range of 1-100, test one value below the range one within the range and one above it.

2. Boundary value Analysis

This technique tests values at and around input limit because defects commonly occur at boundaries.

Example: For a range of 1-100, test 0, 1, 2, 99, 100, and 101.

3. Decision Table Testing

This technique tests combinations of conditions and their expected actions.

Example: An electricity payment should be marked successfully only when the bill and amount are valid and the payment getewey confirms success.

4. State Transition Testing

This technique checks how a system moves blw different states.

Example: An electricity bill may move from pending to processing and then to paid or failed.

5. Risk-Based Testing
This technique prioritizes features that could cause serious problems such as payment processing authentication and bill calculation.

6. Regression Testing
Regression testing repeats previous tests after software changes to ensure that existing features continue to work correctly.

7. Automated Testing
Automated testing uses tools such as selenium, Junit and postman to execute tests efficiently and consistently. this useful for repetitive tests and large applications.

Measuring Testing Effectiveness
Testing effectiveness can be measured using requirement coverage, test pass rate, defect detection rate defect discovered after release and regression test success rate

Requirement coverage is calculated as:

$$\text{coverage} = \frac{\text{Requirements Tested}}{\text{Total Tested Requirements}} \times 100$$

Conclusion:
Software testing should be treated as a systematic engineering activity rather than random execution. A proper process involving requirement analysis test planning, structured test case design, execution defect reporting and evaluation improves testing efficiency.

Techniques such as equivalence partitioning, boundary value analysis decision table testing risk-based testing regression testing and automation improve defect and software reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	